

コードは、あなたが眠っている間に書かれる

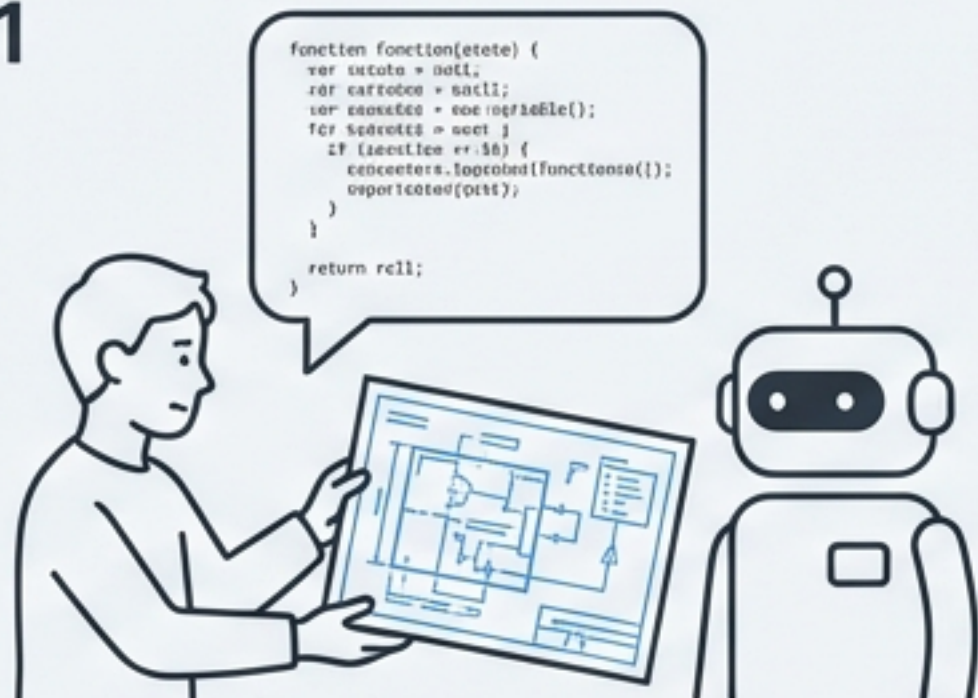
Claude Codeの自律ループエージェント `ralph-wiggum` マスタークラス



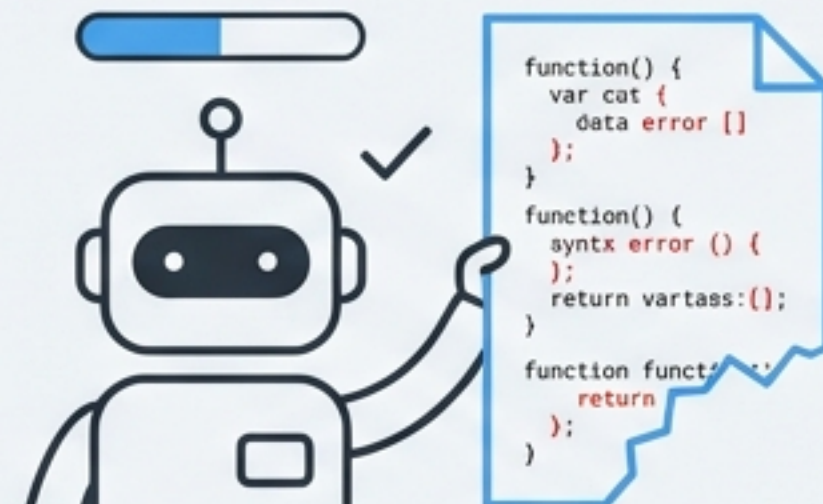
Anthropic公式プラグイン

AIコーディングエージェントの「早すぎる諦め」という課題

1



2



3



複雑なタスクや長時間の作業において、AIはしばしば途中で「完了した」と判断し、処理を終了してしまう。

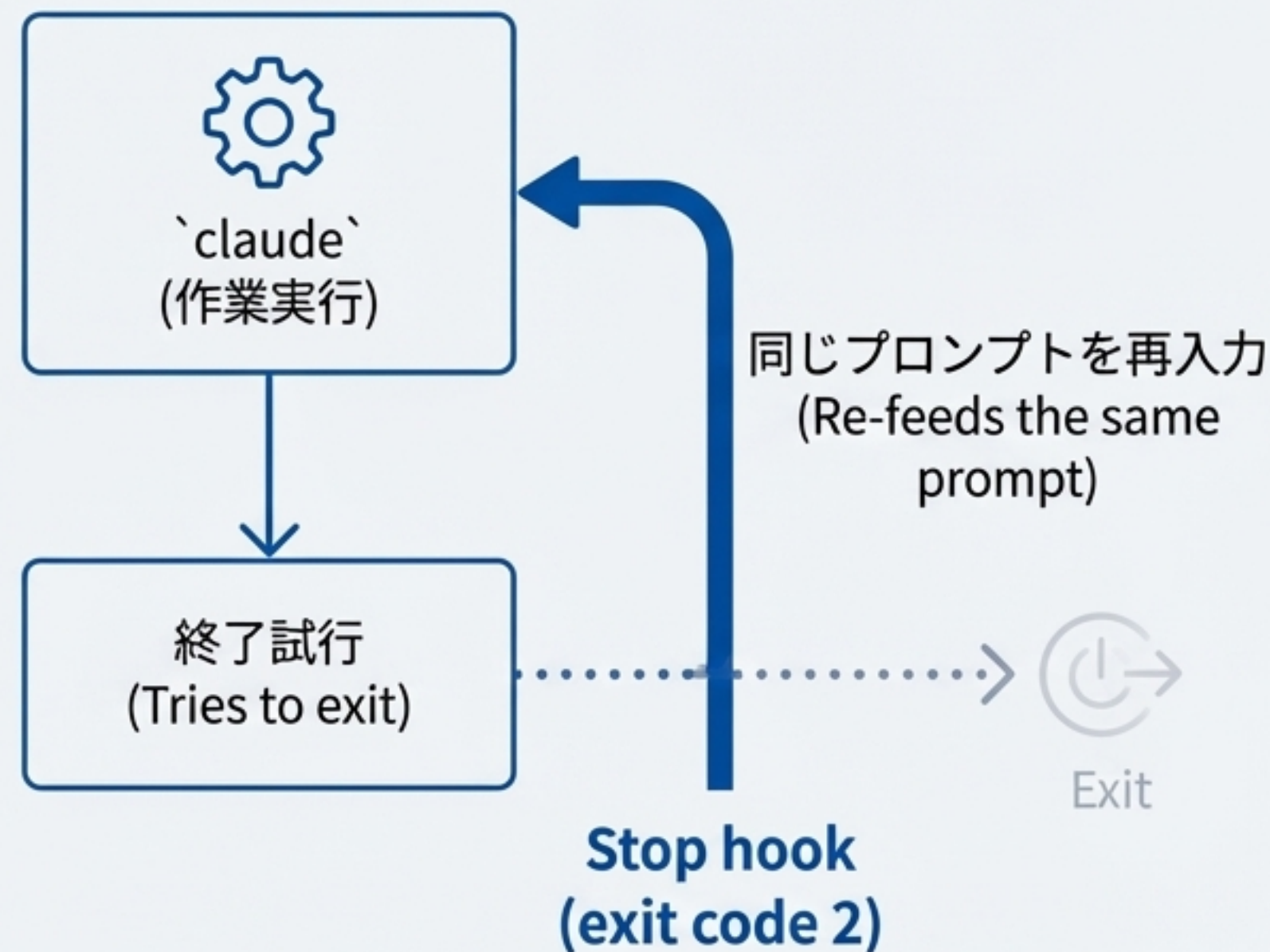
- 結果として、不完全なコードや未修正のエラーが残る。
- 開発者は残作業の修正や手戻りに時間を費やすことになる。
- 「結局、あなたの仕事が増える」
— 生産性向上のためのツールが、逆に手間を増やすジレマ。

解決策は、AIの「停止」をブロックする単純なアイデア

Claude Code公式プラグイン `ralph-wiggum`。シン
プソンのキャラクターにちなんで名付けられた、
粘り強さを体現するツール。

Core Mechanism

- **Stop hook:** Claudeがタスクを完了したと判断し、終了しようとする瞬間に作動。
- **自己参照フィードバックループ (Self-referential feedback loop):** 終了をブロックし、全く同じ初期プロンプトを再入力する。
- **継続的な改善:** 各反復（イテレーション）で、AIは前回の実行で変更されたファイルやGit履歴をコンテキストとして読み込み、自己修正を繰り返す。



注：外部のbashループは不要。セッション内で完結。

失敗をデータに変える開発哲学

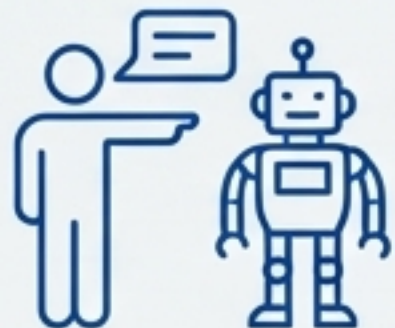
"Ralph is a Bash loop."

— Geoffrey Huntley

Philosophical Principles

- 1** 反復は完璧さに勝る (Iteration > Perfection)
最初から完璧を目指すのではなく、ループを通じて洗練させる。
- 2** 失敗はデータである (Failures Are Data) "決定論的に悪い方が、非決定論的な世界では良い"
失敗は予測可能で有益な情報となり、プロンプトを調整するためのインプットになる。
- 3** 永続性が勝利する (Persistence Wins)
成功するまで試行を続ける。ループがリトライロジックを自動化する。

The Skill Shift



旧 (Old)

AIに各ステップを細かく指示するスキル。



新 (New)

AIが成功に向かって自律的に反復できる
「システム」と「完了条件」を設計するスキル。

自律ループが最も輝くタスク

「完了」をプログラムで検証できる、明確な成功基準を持つタスク。



大規模リファクタリング (Large refactors)

フレームワーク移行、依存関係のアップグレード、APIバージョン変更など。



テストカバレッジ向上 (Test coverage)

「src/内の未カバー関数すべてにテストを追加せよ」といったタスク。



バッチ処理 (Batch operations)

ドキュメント生成、コード標準化、サポートチケットのトリアージ。



グリーンフィールド開発 (Greenfield builds)

一晩でのプロジェクト雛形作成と反復的な改善。

人間の判断が不可欠な領域

`ralph-wiggum`は機械的な実行を自動化するものであり、何を構築すべきかという戦略的判断を自動化するものではない。

Don't Use Autonomous Loops For:

- ✗ 曖昧な要件 (Ambiguous requirements):
「完了」を正確に定義できなければ、ループは収束しない。
- ✗ アーキテクチャ設計 (Architectural decisions):
新しい抽象化や設計思想には、反復ではなく人間の推論が必要。
- ✗ セキュリティ機密性の高いコード (Security-sensitive code):
認証、決済、データ処理などは、各ステップで人間のレビューが必須。
- ✗ 探索的作業 (Exploration):
コードベースの理解には、自動化されたパスではなく、人間の好奇心が必要。



現実世界での驚異的な成果

\$50k → \$297



YCハッカソンにて、本来であれば\$50,000の契約作業に相当する開発を、わずか\$297のAPIコストで完了。一晩で6つ以上のリポジトリを納品。

4分 → 2秒



ある開発者が統合テストから単体テストへの移行をループに任せ、テスト実行時間を4分から2秒へと劇的に短縮。

3ヶ月 → 1言語



Geoffrey Huntleyが単一のプロンプトで3ヶ月間ループを実行し、Gen Zスラング（`slay`=関数, `sus`=変数）を使った完全なプログラミング言語「Cursed」をコンパイラ、標準ライブラリ付きで開発。

プロンプトの技術：成功率を高める4つの原則

`ralph-wiggum`の成功は、モデルの性能だけでなく、オペレーター（開発者）のプロンプト設計スキルに依存する。

1 明確な完了条件

曖昧な指示を避け、テストの成功、特定文字列の出力など、機械的に検証可能なゴールを設定する。

2 インクリメンタルな目標

巨大なタスクはフェーズに分割し、段階的に達成させる。

3 自己修正の組み込み

TDD（テスト駆動開発）のように、テスト→実装→デバッグのサイクルをプロンプト自体に組み込む。

4 安全な脱出路

無限ループを防ぐため、常に反復回数の上限（`--max-iterations`）を設定する。また、行き詰まった場合の報告手順を指示に含める。



実践：悪いプロンプト vs 良いプロンプト

✖ Bad

曖昧で検証不可能 (Ambiguous & Unverifiable)

"Build a todo API and make it good."

"Create a complete e-commerce platform."

"Write code for feature X."

✔ Good

明確で検証可能 (Specific & Verifiable)

"Build a REST API for todos."

When complete:

- All CRUD endpoints working
- Input validation in place
- **Tests passing** (coverage > 80%)
- **Output: COMPLETE**

"**Phase 1:** User authentication (JWT, tests)"

Phase 2: Product catalog (list/search, tests)"

..."

"Implement feature X following TDD:"

1. **Write failing tests**
2. Implement feature
3. Run tests and fix until all green
4. **Output: COMPLETE**

30秒で始める：インストールと基本コマンド

Installation Steps

1. マーケットプレイスを追加

```
/plugin marketplace add anthropics/claude-code
```

2. プラグインをインストール (Install the Plugin)

```
/plugin install ralph-wiggum@claude-code-plugins
```

3. Claude Codeを再起動 (Restart Claude Code)

Core Commands

- ループ開始 (Start a loop)

```
/ralph-loop "<prompt>" --max-iterations  
N --completion-promise "text"
```

- ループ強制停止 (Cancel a loop)

```
/cancel-ralph
```



Pro-Tip: コマンド名は環境によって `/ralph-wiggum:ralph-loop` のように名前空間化されることがあります。`/` を押して `ralph` で絞り込むのが確実です。

安全第一：自律ループのガードレール



**`--max-iterations` を常に設定する
(ALWAYS set `--max-iterations`)**

これは無限ループと予期せぬコスト増を防ぐための最も重要な安全装置です。

Understanding the Tools

`--completion-promise`の限界: このフラグは「完全一致」でのみ機能するため、複数の完了条件や僅かな出力の違いで失敗することがあります。主要な安全策として依存すべきではありません。

Cost Awareness



自律ループはトークンを消費します。大規模なコードベースでの50回の反復は、\$50-\$100以上のAPIクレジットを消費する可能性があります。

必ず保守的な反復回数から始め、使用状況を監視してください。

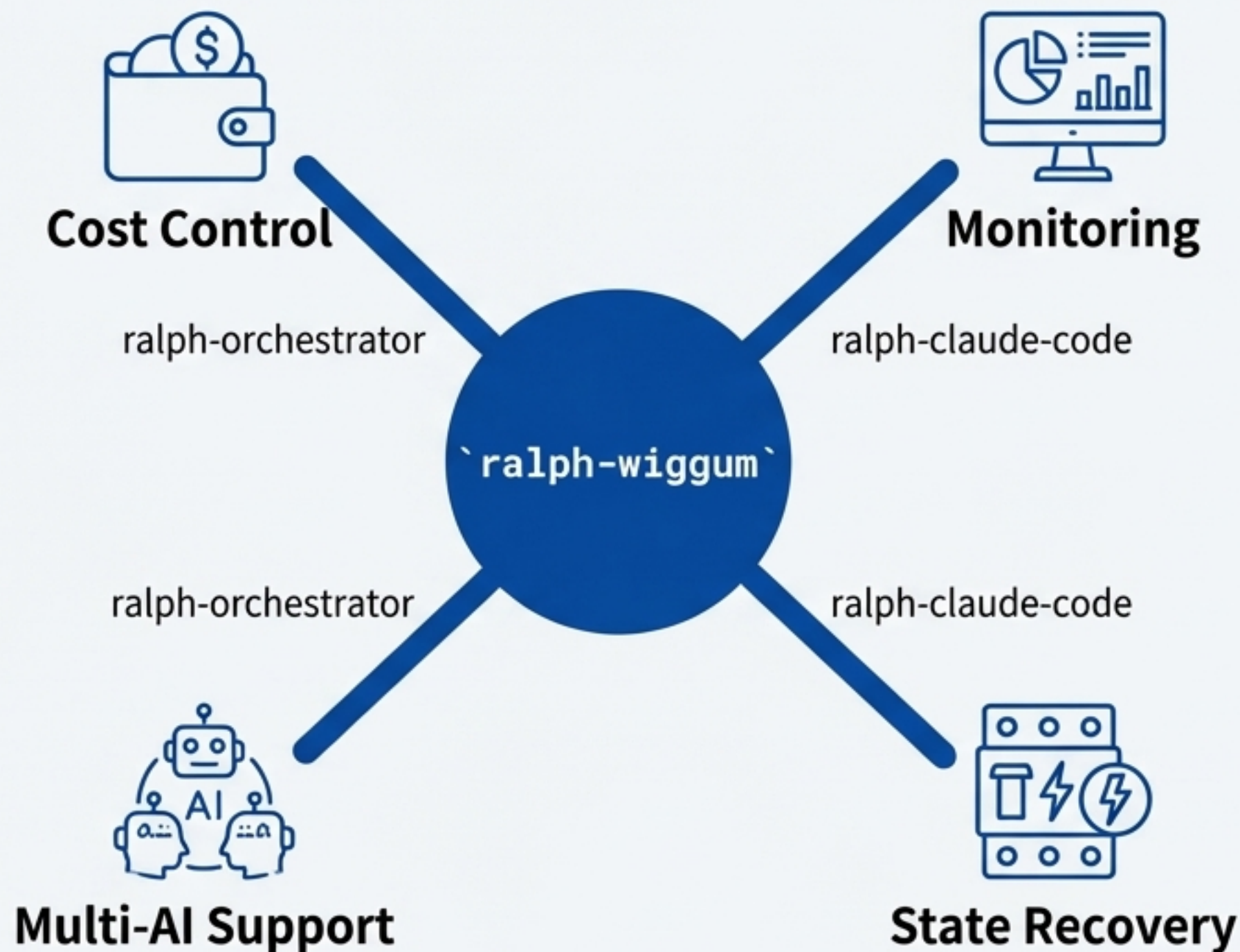
全体像：単一ツールからエコシステムへ

The Paradigm Shift

- **`ralph-wiggum`**は、"SDLC is Collapsing"
(ソフトウェア開発ライフサイクルの崩壊) という大きな変化の一翼を担う。
- 計画、構築、テスト、デプロイといった従来のフェーズが、エージェントによる継続的なフローに溶解していく。

Community-Driven Evolution

- 公式プラグインが提供するのはコアメカニズム。エコシステムが本番用のラッパーを構築している。
- **`ralph-claude-code`**: レート制限、tmuxダッシュボード、障害回復のためのサーキットブレーカーを追加。
- **`ralph-orchestrator`**: トークン追跡、支出制限、Gitチェックポイント、マルチAIサポートなどの運用課題を解決。



プロジェクトの健全性：コミュニティからの静かな信頼



Star: 901 → 903 (+2)

爆発的な成長期ではなく、AIプラグインに関心を持つ開発者層からの注目が静かに集っている「観測フェーズ」にあることを示唆。



Fork: 92 → 92 (Stable)

プロジェクトの利用や派生開発の基盤が安定している。

Conclusion

現在のエンゲージメントは、能動的な貢献者（Fork）よりも受動的な追跡者（Star）を引きつけている。これは、プロジェクトが安定した基盤の上で、将来の採用候補として評価・ブックマークされる「潜在的価値の蓄積期」にあることを意味する。

あなたの最初の自律ループを始めよう

Recap: `ralph-wiggum` は、人間の判断が重い作業ではなく、完了条件が明確な機械的バッチ作業で真価を発揮する。

A Simple First Task

- 1. ****プラグインをインストールする (30秒)****
- 2. ****小さなスコープで始める****:

```
# 例: src/utils/内のエクスポートされた全関数にJSDocコメントを追加  
/ralph-loop "Add JSDoc comments to all exported functions in src/utils/"
```

- 3. ****保守的な反復回数を設定する****:

```
--max-iterations 10
```

- 4. ****完了後、`git diff` をレビューする。 ****

**目を覚ましたら、動くコードがそこにある。
(Wake up to working code.)**